



Universidad Nacional Autónoma de México

Facultad de Ciencias

Sistemas Operativos

Tarea 5: Simulación de administración de memoria

Algoritmos de reemplazo de páginas: Second Chance y Aging

Equipo: E04

Integrantes: David Rivera Morales (320176876)
Yessica Vianney Montes de Oca Aguilar (315050116)

Fecha de entrega: Mayo de 2026

Licenciatura en Ciencias de la Computación

1. Introducción: qué es un fallo de página

La memoria virtual divide el espacio de direcciones de un proceso en **páginas** y la memoria física en **marcos** del mismo tamaño. Mientras el programa corre, solo una parte de sus páginas está cargada en marcos. Cuando el programa referencia una página que no está en memoria se produce un **fallo de página**: el sistema operativo tiene que traerla desde disco y, si no hay marcos libres, **expulsar** alguna página residente para hacerle espacio.

El algoritmo de reemplazo decide cuál página expulsar. Su calidad se mide con la **tasa de fallos** sobre una **cadena de referencias** (la secuencia de páginas que pide el programa):

$$\text{tasa de fallos} = \frac{\text{fallos de página}}{\text{número de referencias}}$$

Las referencias reales tienen **localidad**: un programa tiende a usar un conjunto reducido de páginas durante un intervalo y luego cambia a otro. Por eso los algoritmos que recuerdan qué páginas se usaron recientemente suelen ganar, y por eso hace falta un algoritmo de reemplazo en vez de expulsar cualquier página al azar.

Un ejemplo pequeño fija la idea. Con 3 marcos y la cadena `7 0 1 2 0 3`, las primeras cuatro referencias son fallos (los marcos empiezan vacíos y luego hay que expulsar), la quinta es un acierto porque la página 0 sigue residente, y la sexta vuelve a ser fallo:

Paso	Ref	Marcos	Resultado
1	7	7, –, –	fallo (marco libre)
2	0	7, 0, –	fallo (marco libre)
3	1	7, 0, 1	fallo (marco libre)
4	2	2, 0, 1	fallo, expulsa a 7
5	0	2, 0, 1	acierto
6	3	2, 0, 3	fallo, expulsa a 1

Tabla 1: Ejemplo manual con 3 marcos y reemplazo FIFO.

Todo el trabajo consiste en responder una sola pregunta: al llegar el paso 4 (o el 6), ¿cómo se elige la víctima? Implementamos un simulador en C11 de reemplazo de páginas con los dos algoritmos que nos tocaron, **Second Chance** y **Aging**, y usamos FIFO, LRU y OPT como líneas base de comparación. La selección cubre el mínimo de complejidad de 4 puntos para un equipo de dos integrantes.

2. El problema de aproximar LRU

LRU (Least Recently Used) expulsa la página usada hace más tiempo. Es una política buena porque la localidad hace que el pasado reciente sea un buen predictor del futuro cercano, pero mantener el orden exacto de uso es caro: cada referencia obliga a actualizar la estructura que guarda ese orden, no solo los fallos. FIFO es lo contrario: cuesta casi nada (una cola circular), pero expulsa la página más antigua sin importar si se usa todo el tiempo, así que su tasa de fallos es mala.

Entre los dos extremos viven las aproximaciones. Second Chance y Aging son dos formas distintas de estimar "qué página está menos viva" con poca metadata:

Algoritmo	Metadata por marco	Idea	Aproxima a
Second Chance	1 bit de referencia	Reloj circular con segunda oportunidad	LRU / FIFO híbrido
Aging	N bits (8 por defecto)	Contador que envejece y pesa las referencias recientes	LRU con historia finita
FIFO	0 bits extra	Expulsa la página más antigua	–
LRU	Orden de uso	Expulsa la usada hace más tiempo	– (exacto)
OPT	Conocimiento del futuro	Expulsa la que tardará más en volver	– (óptimo teórico)

Tabla 2: Metadata e idea de cada algoritmo.

Los dos algoritmos son complementarios: Second Chance usa un solo bit por marco y es muy barato, mientras que Aging usa un contador de N bits por marco y consigue una aproximación más fina a LRU. Compararlos permite discutir el compromiso entre metadata, costo y tasa de fallos.

Dos conceptos aparecen en el análisis:

- **Anomalía de Belady:** con FIFO, aumentar el número de marcos puede *aumentar* el número de fallos. Ocurre porque FIFO no es un algoritmo de pila (*stack algorithm*).
- **Propiedad de pila:** si un algoritmo la cumple, los fallos con F marcos son un superconjunto de los fallos con $F + 1$ marcos. LRU y OPT la cumplen; FIFO no. Second Chance tampoco la cumple (la sección de resultados muestra la anomalía); Aging no es de pila en sentido estricto, aunque en nuestras trazas no mostró la anomalía.

3. Second Chance (reloj)

3.1. Idea

Se mantienen los marcos en una lista circular y un puntero `hand`. Cada marco tiene un **bit de referencia R**:

- $R = 1$ significa "esta página se usó desde la última vez que la miré".
- $R = 0$ significa "no se usó recientemente; es candidata a salir".

En cada referencia:

1. Si la página está en memoria (acierto), se pone $R = 1$.
2. Si no está (fallo):
   - se examina el marco apuntado por `hand`;
   - si $R = 1$, se le da una segunda oportunidad: se limpia a $R = 0$ y se avanza el puntero;
   - si $R = 0$, ese marco es la víctima; se carga la página nueva con $R = 1$ y se avanza el puntero.
3. Si hay marcos libres se usa uno de ellos y también se avanza el puntero.

Pseudocódigo:

```
si p está en memoria:
    R[p] = 1
si no:
    mientras R[hand] == 1:
        R[hand] = 0
        hand = (hand + 1) mod F
    víctima = memoria[hand]
    memoria[hand] = p
```

```
    R[hand] = 1
    hand = (hand + 1) mod F
```

3.2. Ejemplo paso a paso

La tabla siguiente muestra las primeras 8 referencias de la traza clásica con 4 marcos. En el estado, `página:R` es el contenido de cada marco.

Paso	Ref	Resultado	Estado	Víctima
1	7	fallo	`7:1 - - -`	–
2	0	fallo	`7:1 0:1 - -`	–
3	1	fallo	`7:1 0:1 1:1 -`	–
4	2	fallo	`7:1 0:1 1:1 2:1`	–
5	0	acierto	`7:1 0:1 1:1 2:1`	–
6	3	fallo	`3:1 0:0 1:0 2:0`	7
7	0	acierto	`3:1 0:1 1:0 2:0`	–
8	4	fallo	`3:1 0:0 4:1 2:0`	1

Tabla 3: Second Chance, traza clásica, 4 marcos.

Los pasos 1 a 4 llenan los marcos: cada fallo usa un marco libre, deja su bit en 1 y avanza la mano. En el paso 5 la página 0 vuelve a referenciarse; como ya estaba, es un acierto y su bit queda en 1 (ya lo estaba).

El paso 6 es el interesante. Llega la página 3, no está, y la memoria está llena. El reloj recorre los cuatro marcos (5 visitas contando la vuelta final), encuentra todos los bits en 1, los limpia (4 bits limpiados) y termina expulsando la página 7, que es donde comenzó el recorrido. La página 0, que se usó en el paso 5, conserva su segunda oportunidad solo hasta que el reloj la vuelve a mirar; en esta corrida su bit se limpia igual que los demás.

En el paso 7 la página 0 da un acierto, así que su bit sube de nuevo a 1. En el paso 8 llega la página 4: la mano está sobre la página 0, le limpia el bit (un solo bit limpiado), avanza a la página 1, la encuentra en 0 y la expulsa. El mismo acierto del paso 7 protegió a la página 0 una vuelta más.

3.3. Comportamiento y costo

Si todos los bits están en 1 al buscar víctima, el reloj los limpia y expulsa el marco donde comenzó el recorrido. Si esto sucede en cada reemplazo, avanza como FIFO. Tras segundas oportunidades anteriores, la mano no necesariamente señala la página con la fecha de carga más antigua, así que el algoritmo no equivale a FIFO en general. Second Chance puede sufrir la anomalía de Belady; la sección de resultados muestra un caso (9 fallos con 3 marcos, 10 con 4).

El costo es un bit por marco y un puntero. Cada fallo puede recorrer hasta F marcos distintos ($F+1$ visitas si da la vuelta completa y vuelve al primero); en el caso promedio de nuestras trazas examinó entre 2 y 3 marcos por fallo con memoria llena.

4. Aging (envejecimiento)

4.1. Idea

Cada marco tiene un **contador de N bits** (por defecto $N = 8$). El contador resume las últimas N referencias de la página: el bit más significativo (MSB) pesa más que los bits viejos.

Convención de esta simulación. El Aging habitual muestrea periódicamente el bit de referencia R, calcula `(age » 1) | (R « (N-1))` y limpia R; es la forma que describe Tanenbaum, con un reloj de muestreo cada k ticks. Nosotros usamos una simplificación: un paso de envejecimiento por referencia, desplazamiento antes de seleccionar víctima e inserción del uso actual al final. No hay reloj de muestreo independiente ni parámetro para agrupar varias referencias por intervalo. Todos los resultados corresponden a esta convención; no deben compararse directamente con otra frecuencia u orden de actualización.

En cada referencia:

1. Se desplaza a la derecha **todos** los contadores residentes: `age = age » 1`. Así cada referencia sin usar la página reduce su peso.
2. Si la página está en memoria (acierto), se pone su MSB en 1: `age = age | MSB`.
3. Si no está (fallo), se elige como víctima la página con **menor contador**. Los empates se resuelven por el menor índice de marco. La página nueva se carga con su MSB en 1.

Pseudocódigo:

```
para cada página residente q:
    age[q] = age[q] >> 1

si p está en memoria:
    age[p] = age[p] | MSB
si no:
    víctima = argmin(age[q])      // empate: menor índice de marco
    memoria[víctima] = p
    age[p] = MSB
```

Una página que se usa por primera vez (o cuyo contador estaba en 0) queda con `1000...`; si se vuelve a usar, acumula bits altos (`1100...`, `1110...`). Una página que deja de usarse va perdiendo bits (`0100...`, `0010...`, ...) y en N referencias queda en 0. Es una versión ponderada de LRU: no recuerda el instante exacto del último uso, sino cuántas veces y qué recientemente se usó dentro de una ventana de N referencias.

4.2. Ejemplo paso a paso

La misma traza clásica con 4 marcos. En el estado, `página:edad` con la edad en hexadecimal.

Paso	Ref	Resultado	Estado	Víctima
1	7	fallo	`7:80 - - -`	–
2	0	fallo	`7:40 0:80 - -`	–
3	1	fallo	`7:20 0:40 1:80 -`	–
4	2	fallo	`7:10 0:20 1:40 2:80`	–
5	0	acierto	`7:08 0:90 1:20 2:40`	–
6	3	fallo	`3:80 0:48 1:10 2:20`	7 (edad `04`)
7	0	acierto	`3:40 0:A4 1:08 2:10`	–
8	4	fallo	`3:20 0:52 4:80 2:08`	1 (edad `04`)

Tabla 4: Aging con 8 bits, traza clásica, 4 marcos.

Aquí se ve el envejecimiento en acción. En el paso 5 la página 0 se usa otra vez: todos los contadores se desplazan (`80`→`40`, `40`→`20`, `20`→`10`) y el de la página 0 además recibe el MSB, así que sube a `90` y queda protegida.

En el paso 6 los contadores se desplazan otra vez: `7:08`→`04`, `0:90`→`48`, `1:20`→`10`, `2:40`→`20`. La víctima es la página 7, cuyo contador `04` es el mínimo; la página 3 se carga con `80`. Notar que

la página 0 no vuelve a marcarse en el paso 6: su bit alto del paso 5 ya venía incluido y solo se desplaza.

En el paso 8 la víctima es la página 1, porque su contador `04` es el mínimo después del envejecimiento. Ambos algoritmos aciertan en los pasos 5 y 7, pero por caminos distintos: Second Chance perdona por bit y Aging ordena por contador.

4.3. Comportamiento, bits y costo

El parámetro clave es N. Con N chico el algoritmo olvida rápido y se aleja de LRU; con N grande se acerca a LRU a cambio de más metadata. Más bits significan una ventana de historia más larga: con 8 bits solo se recuerdan las últimas 8 referencias, con 16 bits las últimas 16. El simulador permite cambiarlo con `-b` y la sección de resultados mide ese efecto.

El costo es N bits por marco (8 bits = 1 byte por marco en la configuración por defecto). En cada referencia hay que desplazar F contadores y, en cada fallo, recorrer F marcos para buscar el mínimo: $O(F)$ por referencia. Para las memorias pequeñas de esta simulación el costo es despreciable, pero es mayor que el de Second Chance.

5. Líneas base: FIFO, LRU y OPT

- **FIFO:** cola circular; expulsa la página cargada hace más tiempo. No mira referencias. Es la línea base más simple y la que exhibe la anomalía de Belady.
- **LRU:** expulsa la página con el `last_used` más antiguo. Es exacto, pero requiere mantener el orden de uso (en la simulación se guarda un contador por marco). Sirve para medir cuánto se acercan Second Chance y Aging al ideal práctico.
- **OPT (Belady):** conoce la cadena completa y expulsa la página cuyo próximo uso está más lejos (o que no vuelve a usarse). Es imposible de implementar en un sistema real, pero da la cota inferior de fallos y permite interpretar cuánto margen queda.

6. Metodología

6.1. Simulación dirigida por trazas

La simulación es *trace-driven*: el tiempo es discreto y cada elemento de la cadena de referencias es un paso. El estado es:

```
F marcos: pagina residente, bit R, contador de edad, paso de carga,
          ultimo uso
hand: posicion del reloj (Second Chance y FIFO)
estadisticas: fallos, aciertos, expulsiones, cargas libres, etc.
```

En cada paso se busca la página en los marcos; si está, es acierto y se actualiza el metadata (R, edad o último uso); si no está, es fallo y se usa un marco libre o se elige víctima según el algoritmo. Los empates se resuelven siempre por el menor índice de marco para que las corridas sean deterministas.

6.2. Trazas de prueba

Archivo	Referencias	Páginas	Propósito
`clasica.txt`	20	6	Caso clásico de Silberschatz; páginas distintas: 0, 1, 2, 3, 4, 7
`belady.txt`	12	5	Anomalía de Belady (FIFO 9→10 fallos al pasar de 3 a 4 marcos)
`ciclica.txt`	100	10	Peor caso: páginas 0..9 en ciclo; el conjunto de trabajo no cabe con menos de 10 marcos
`localidad.txt`	2000	24	Carga principal: 8 fases de 250 referencias, conjunto caliente de 6 páginas con 85 % de probabilidad, desplazado 3 páginas por fase
`mixta.txt`	1200	20	Segunda carga: 6 fases, conjunto caliente de 5 páginas con 70 %, 30 % uniforme

Tabla 5: Las cinco cadenas de referencia.

Las trazas se generan con `scripts/gen_traces.py` usando semillas fijas; el simulador no usa aleatoriedad, así que dos corridas con el mismo archivo y los mismos parámetros dan exactamente el mismo resultado. La traza de localidad es la más importante para comparar algoritmos porque se parece a un programa real: hay un conjunto de páginas que se reutiliza durante una fase y luego cambia. La traza mixta sirve para comprobar que las conclusiones no dependen de una sola elección de parámetros.

6.3. Métricas

Medimos fallos, aciertos, tasa de fallos, expulsiones (fallos con memoria llena), cargas libres y residencia media (pasos entre carga y expulsión). Para Second Chance contamos además los bits limpiados, los marcos examinados y los avances de mano; para Aging, los empates en el mínimo y la edad media de la víctima.

6.4. Estructura del simulador

El motor está en `src/paging.c`, usa arreglos estáticos (hasta 64 marcos, 200 000 referencias, 4096 páginas) y no imprime nada: devuelve eventos y acumula estadísticas. Este es el caso de Second Chance dentro de `sim_step`, tal como está implementado:

```
int scans = 0;
int cleared = 0;
while (s->fr[s->hand].ref) {
    s->fr[s->hand].ref = 0;
    cleared++;
    scans++;
    s->st.ref_cleared++;
    s->st.hand_steps++;
    s->hand = (s->hand + 1) % s->frames;
}
scans++;
idx = s->hand;
ev.victim = s->fr[idx].page;
s->st.scans += scans;
s->st.hand_steps++;
s->hand = (s->hand + 1) % s->frames;
```

El lazo limpia bits hasta encontrar un marco con $R = 0$; ese marco es la víctima. Los contadores `scans` y `cleared` no son necesarios para el resultado, pero permiten analizar el costo del reloj.

6.5. Validación

Validamos el motor de tres formas:

- **24 self-tests** clásicos (traza de Silberschatz con 3 y 4 marcos, anomalía de Belady, vuelta completa del reloj, decaimiento del contador y estrés). Salida: `24 comprobaciones, 0 fallos`. Los valores coinciden con los del libro: con 3 marcos, FIFO = 15, LRU = 12 y OPT = 9.
- **Verificación cruzada**: una implementación independiente en Python (`scripts/referencia.py`) corre las mismas trazas; el script de verificación genera 30 trazas aleatorias, prueba de 1 a 8 marcos y compara fallos de los cinco algoritmos (1440 comparaciones, 0 diferencias), más 13 regresiones del lector de trazas.
- **Invariantes** al final de cada corrida: fallos + aciertos = referencias, expulsiones = fallos − cargas libres, sin páginas duplicadas en memoria, y en Second Chance la igualdad `marcos examinados = bits limpiados + fallos con memoria llena`. Si alguna falla, el programa termina con error en vez de imprimir resultados dudosos.

7. Resultados y cómo leerlos

La configuración por defecto es 4 marcos y 8 bits para Aging.

7.1. Comparación principal (4 marcos)

Traza	Refs.	Second Chance	Aging	FIFO	LRU	OPT
`clasica`	20	9 (45.00 %)	**8 (40.00 %)**	10 (50.00 %)	**8 (40.00 %)**	**8 (40.00 %)**
`belady`	12	10 (83.33 %)	**8 (66.67 %)**	10 (83.33 %)	**8 (66.67 %)**	**6 (50.00 %)**
`ciclica`	100	100 (100 %)	100 (100 %)	100 (100 %)	100 (100 %)	**70 (70.00 %)**
`localidad`	2000	982 (49.10 %)	**945 (47.25 %)**	996 (49.80 %)	**945 (47.25 %)**	**586 (29.30 %)**
`mixta`	1200	660 (55.00 %)	**638 (53.17 %)**	673 (56.08 %)	**638 (53.17 %)**	**406 (33.83 %)**

Tabla 6: Fallos de página con 4 marcos; tasa entre paréntesis.

Qué significa: en las dos trazas con localidad, Aging empata con LRU y supera a Second Chance (37 fallos menos en `localidad`, 3.8 %; 22 menos en `mixta`, 3.3 %). Second Chance supera a FIFO en ambas, pero la mejora es pequeña (14 y 13 fallos). En la traza cíclica todos los algoritmos que no miran el futuro fallan siempre, como era de esperar. OPT queda por debajo o igual en todas; en las trazas con localidad la diferencia es grande (586 contra 945), y es una cota inferior, no una alternativa implementable.

Métricas detalladas de `localidad` con 4 marcos:

Algoritmo	Fallos	Aciertos	Tasa	Expulsiones	Cargas libres	Residencia media
Second Chance	982	1018	49.10 %	978	4	8.14
Aging	945	1055	47.25 %	941	4	8.45
FIFO	996	1004	49.80 %	992	4	8.03
LRU	945	1055	47.25 %	941	4	8.45
OPT	586	1414	29.30 %	582	4	13.60

Tabla 7: Detalle de la traza `localidad`, 4 marcos.

Qué significa: las páginas de OPT viven casi el doble en memoria (13.6 pasos de residencia media contra 8.45 de Aging); esa es la diferencia entre conocer el futuro y aproximarlo. LRU y Aging dan métricas idénticas porque en esta traza eligen las mismas víctimas.

Costo específico de los algoritmos seleccionados en esa misma corrida:

Algoritmo	Metadata	Bits limpiados	Marcos examinados	Empates	Edad media de víctima
Second Chance	1 bit/marco	1293	2271	–	–
Aging	8 bits/marco	–	–	1	5.16

Tabla 8: Costo medido en `localidad`, 4 marcos.

Qué significa: Second Chance examinó 2271 marcos para 978 fallos con memoria llena, es decir 2.32 visitas por fallo; el reloj casi nunca da la vuelta completa. Aging desplaza los contadores en cada referencia y recorre los marcos en cada expulsión, trabajo adicional que el programa no instrumenta por completo; los 37 fallos evitados son un resultado de la simulación, no una medida de velocidad. No medimos tiempo de CPU.

7.2. Barrido de marcos

Marcos	Second Chance	Aging	FIFO	LRU	OPT
3	1207	1189	1209	1189	811
4	982	945	996	945	586
5	788	754	810	752	422
6	620	561	668	561	305
8	400	385	481	316	194
9	329	356	409	259	164
10	268	327	351	231	141
12	195	276	266	175	107

Tabla 9: Fallos en `localidad` (2000 referencias) según el número de marcos.

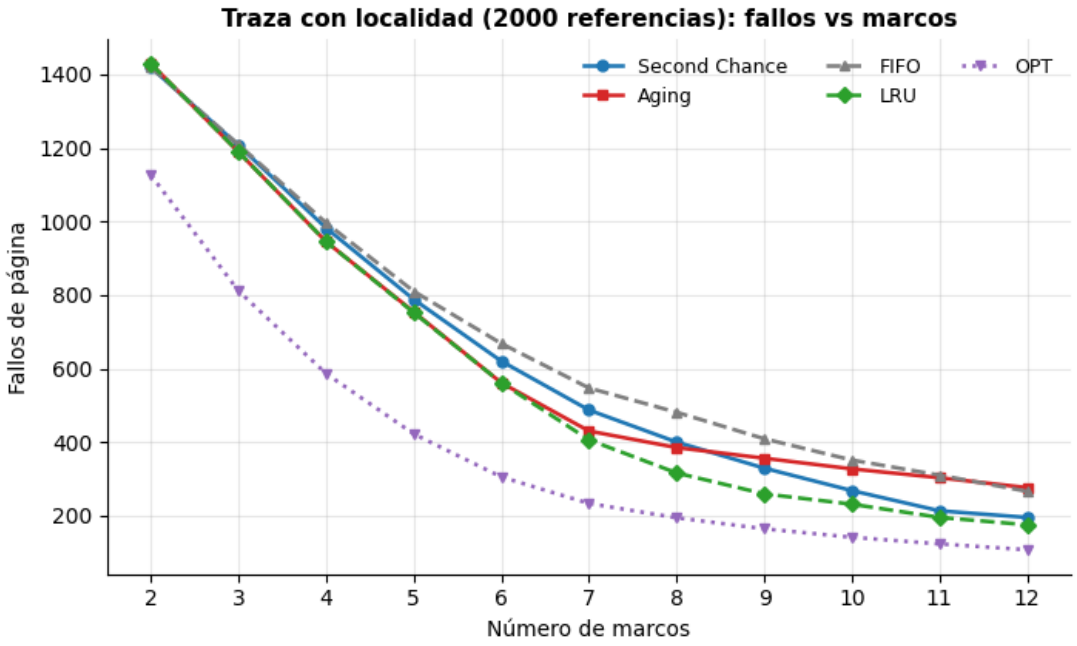


Figura 1: Fallos contra marcos en la traza `localidad`.

Qué significa: hay un cambio de régimen. Con 4 y 6 marcos, Aging empata con LRU y es el mejor de los dos seleccionados. A partir de 9 marcos, Second Chance lo supera, y con 10 o 12 la diferencia crece. La razón está en el ancho del contador: con 8 bits, Aging solo recuerda las últimas 8 referencias, y la cadena no cambia al agregar marcos. Con mayor capacidad pueden

permanecer páginas cuyo último uso quedó fuera de la ventana del contador; varias llegan a cero y el desempate por índice pierde el orden de recencia que LRU sí conserva. Second Chance no tiene ese límite de historia.

La traza `mixta` muestra el mismo patrón:

Marcos	Second Chance	Aging	FIFO	LRU	OPT
3	769	765	770	765	534
4	660	638	673	638	406
5	569	542	589	538	313
6	482	444	507	442	249
8	337	320	375	283	163
9	262	290	311	230	136
10	214	273	277	186	113
12	153	192	209	142	79

Tabla 10: Fallos en `mixta` (1200 referencias) según el número de marcos.

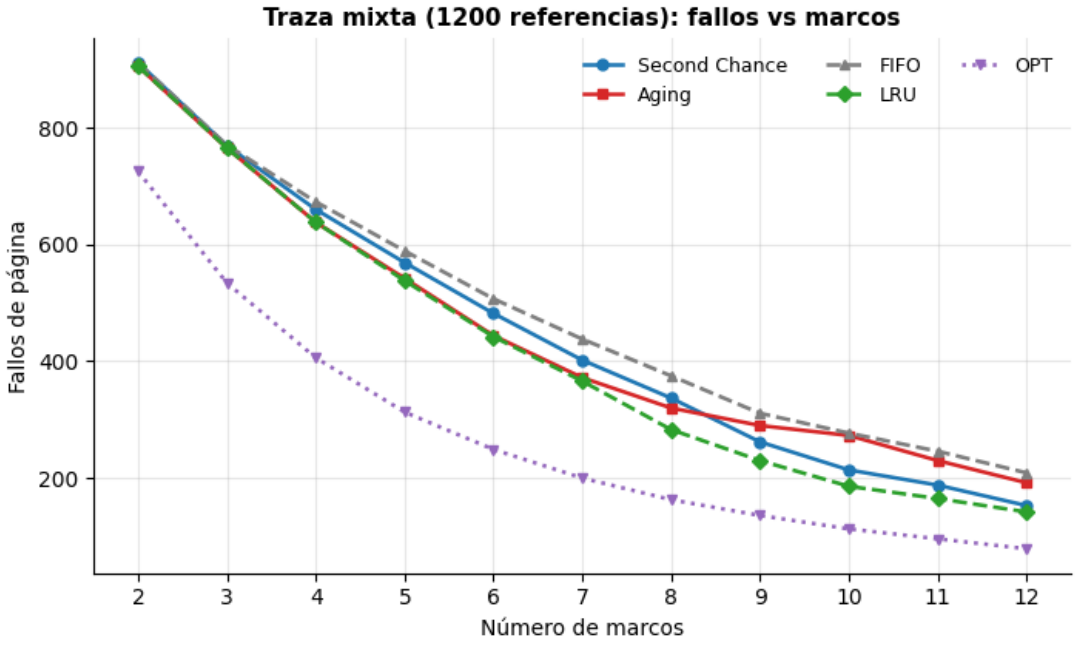


Figura 2: Fallos contra marcos en la traza `mixta`.

Qué significa: el cruce entre Aging y Second Chance se repite alrededor de 9 marcos, lo que indica que la conclusión no es un artefacto de una sola traza.

La traza clásica permite ver el efecto en memoria muy pequeña:

Marcos	Second Chance	Aging	FIFO	LRU	OPT
2	15	17	15	17	13
3	14	12	15	12	9
4	9	8	10	8	8
5	9	7	9	7	7
6	6	6	6	6	6

Tabla 11: Fallos en `clasica` según el número de marcos.

Qué significa: con 2 marcos, LRU y Aging fallan *más* que FIFO y Second Chance (17 contra

15): en memorias muy pequeñas, recordar el último uso puede ser contraproducente si el patrón es cíclico. A partir de 3 marcos, Aging y LRU vuelven a ser los mejores entre los que no conocen el futuro. Aproximar LRU no garantiza ganar en todos los tamaños.

La traza cíclica tiene un comportamiento de acantilado:

Marcos	Second Chance	Aging	FIFO	LRU	OPT
2–8	100	100	100	100	90 → 30
9	100	**65**	100	100	20
10	10	10	10	10	10

Tabla 12: Fallos en `ciclica` según el número de marcos.

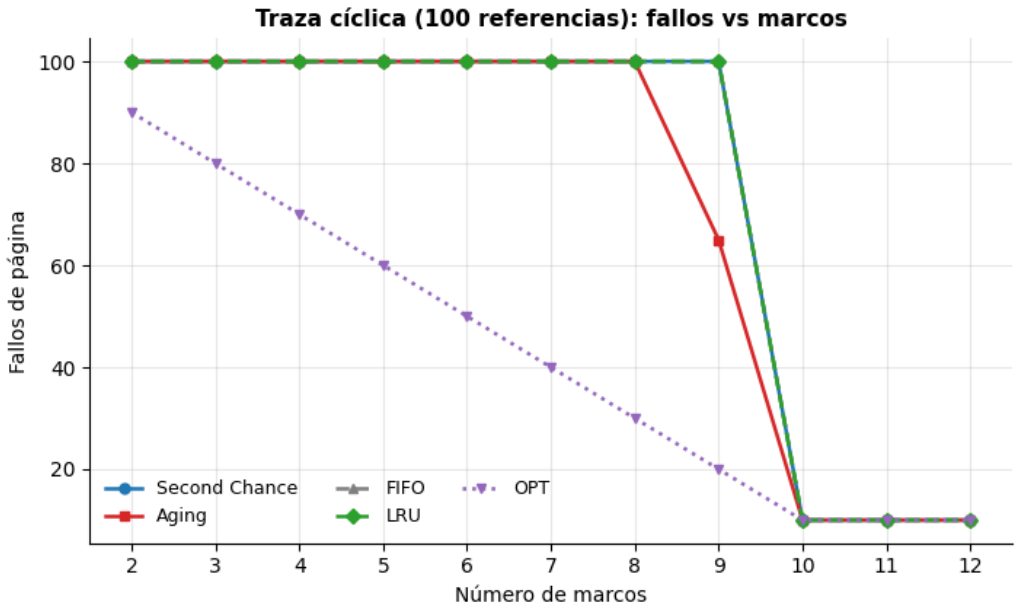


Figura 3: Fallos contra marcos en la traza `ciclica`.

Qué significa: con 2 a 8 marcos, las cuatro políticas en línea fallan en cada referencia. Con 9 marcos, FIFO, LRU y Second Chance caen en el peor caso del ciclo de longitud $F + 1$: expulsan justo la página que se va a necesitar en la siguiente referencia. Aging, al elegir por contador y no por antigüedad, rompe el ciclo y baja a 65 fallos. Con 10 marcos todo cabe y todos bajan a 10. Es un caso límite llamativo, pero aislado: una consecuencia de la interacción entre el ciclo y el contador, no una garantía general.

7.3. Anomalía de Belady

Algoritmo	3 marcos	4 marcos	¿Anomalía?
Second Chance	9	10	**Sí**
Aging	10	8	No
FIFO	9	10	**Sí**
LRU	10	8	No
OPT	7	6	No

Tabla 13: Anomalía de Belady: fallos con 3 contra 4 marcos.

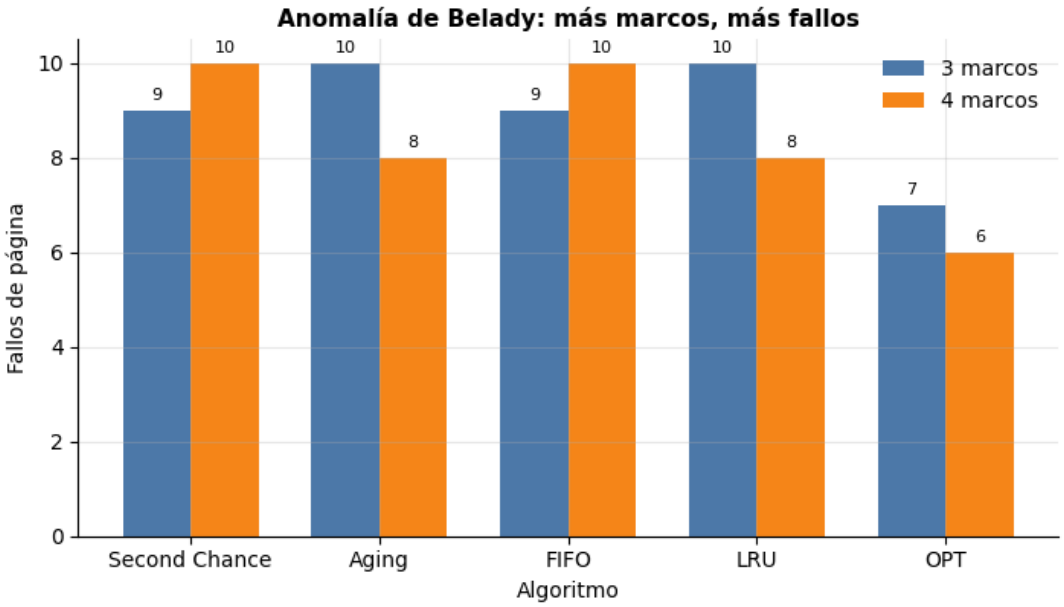


Figura 4: Anomalía de Belady: las barras de Second Chance y FIFO suben al pasar de 3 a 4 marcos.

Qué significa: Second Chance exhibe la anomalía (9 fallos con 3 marcos, 10 con 4) y en esta traza coincide con FIFO. Una vuelta completa del reloj limpia todos los bits y expulsa el marco donde empezó el recorrido, que es el comportamiento de FIFO. LRU y OPT no la exhiben, como corresponde a los algoritmos de pila; Aging, que no es de pila en sentido estricto, tampoco la muestra aquí porque no ordena por antigüedad de carga sino por contador.

7.4. Sensibilidad del contador de Aging

Marcos	LRU	4 bits	6 bits	8 bits	10 bits	12 bits	14 bits	16 bits
6	561	802	586	**561**	561	566	561	561
8	316	692	483	385	350	340	330	327
10	231	540	360	327	278	276	257	254
12	175	434	290	276	260	243	217	219

Tabla 14: Fallos de Aging en `localidad` según los bits del contador; LRU como referencia.

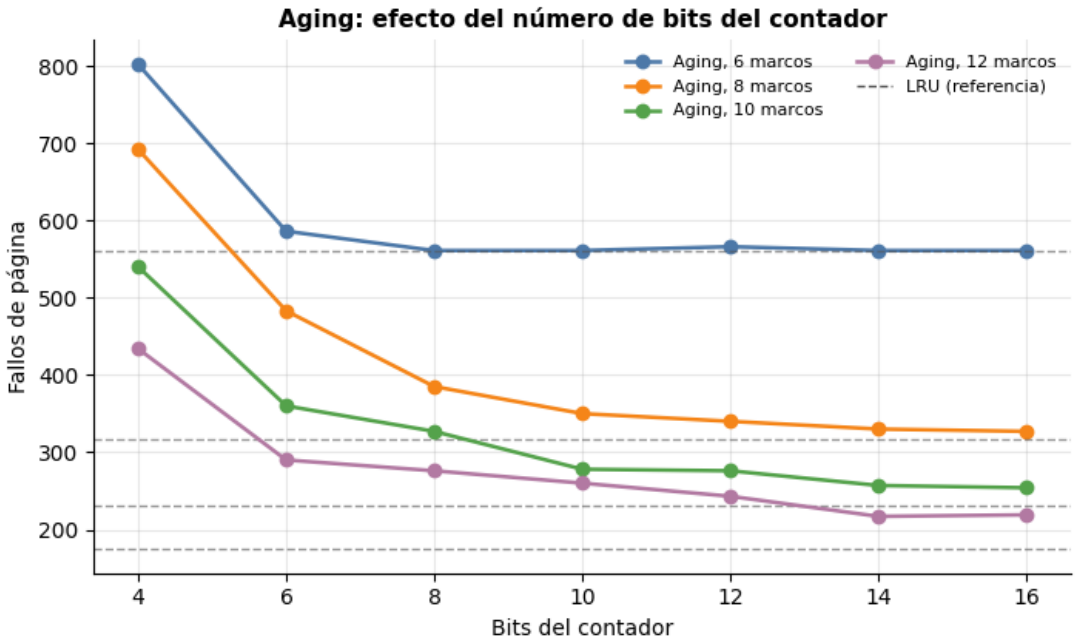


Figura 5: Sensibilidad de Aging al número de bits del contador.

Qué significa: más bits = más historia = más cerca de LRU. Con 8 bits y 6 marcos, Aging ya iguala a LRU (561). Con 8 marcos, 14–16 bits lo acercan (330–327 contra 316), y con 12 marcos todavía queda una brecha (219 contra 175). La pequeña no monotonía (12 bits da 566 en 6 marcos) se debe a los empates; no cambia la conclusión. En el código, el ancho solo modifica la ventana de historia, no la memoria reservada, porque todos los anchos usan el mismo campo `unsigned`.

7.5. Fallos acumulados

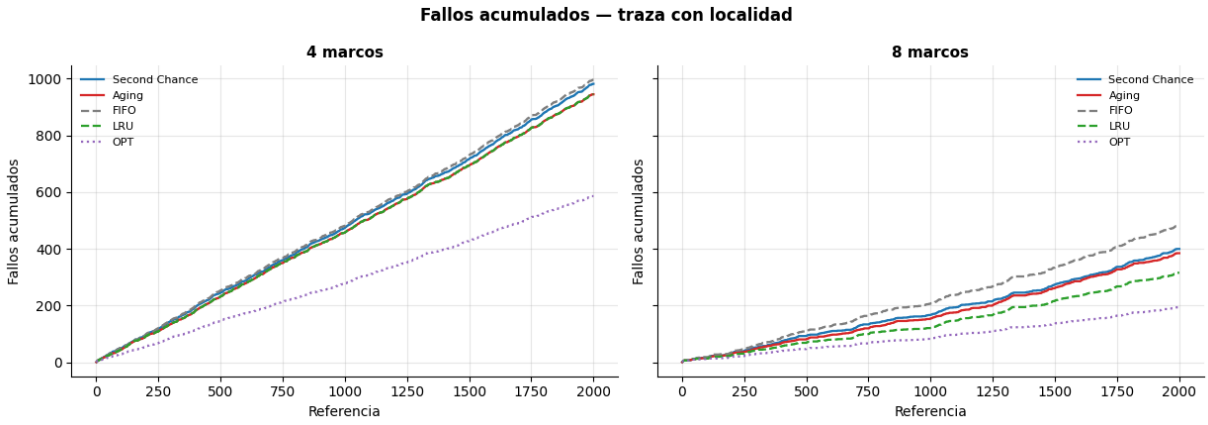


Figura 6: Fallos acumulados en `localidad` con 4 y 8 marcos.

Qué significa: con 4 marcos las curvas de Second Chance, Aging, LRU y FIFO están muy cerca; la diferencia se acumula de a poco. Con 8 marcos la separación es más visible: Aging queda entre Second Chance y LRU, y FIFO es el peor. La pendiente cambia por fase: cuando el conjunto caliente se desplaza, todos los algoritmos sufren un tramo de fallos hasta adaptarse.

8. Análisis

1. Aging es el mejor de los dos seleccionados con la configuración principal. Con 4 marcos empata con LRU en `localidad` y `mixta` (945 y 638 fallos) y supera a Second Chance por 37 y 22 fallos (3.8 % y 3.3 % menos). La mejora es modesta, pero consistente en dos trazas distintas.

2. Second Chance es más barato. Un bit por marco, un puntero y un recorrido solo cuando hay fallo: 2.32 visitas por fallo en `localidad`. Aging necesita 8 bits por marco, desplaza todos los contadores en cada referencia y recorre todos los marcos en cada expulsión. Si el presupuesto de metadata o de CPU es ajustado, Second Chance es la opción razonable; si se puede pagar, Aging da menos fallos.

3. El tamaño de la memoria cambia el ganador. A partir de 9 marcos, Second Chance empieza a ganarle a Aging en `localidad` y `mixta`: con 8 bits la historia es demasiado corta para memorias grandes. Aumentar a 14–16 bits recupera buena parte de la diferencia (330–327 contra 316 con 8 marcos; 257–254 contra 231 con 10). No alcanza con decir "Aging aproxima LRU"; hay que decir con cuántos bits y para qué tamaño de memoria.

4. Second Chance nunca es peor que FIFO en estas trazas. En el peor caso empata (traza cíclica, memoria insuficiente) y en las trazas con localidad mejora. La segunda oportunidad añade un bit lógico por marco y recorridos del reloj; no evita la anomalía de Belady ni garantiza superar siempre a FIFO.

5. Aging no sufre la anomalía de Belady en estas trazas, pero no es un algoritmo de pila. No se puede prometer que siempre baje al agregar marcos; lo que observamos es que el contador elige víctimas distintas de FIFO y evita el peor caso del ciclo con $F = 9$ (65 fallos contra 100 de FIFO, LRU y Second Chance). Es un caso límite llamativo, pero aislado.

6. OPT marca el margen de mejora. En `localidad` con 4 marcos, OPT hace 586 fallos contra 945 de Aging: todavía hay un 38 % de margen teórico. Ese margen no es alcanzable en línea porque OPT necesita conocer el futuro; sirve para dimensionar cuánto se está perdiendo con una aproximación.

7. El patrón de reutilización importa. La traza cíclica reutiliza cada página cada 10 referencias, pero con 4 marcos ninguna política en línea retiene las páginas hasta su siguiente uso. Que el conjunto de trabajo no quepa no obliga a cualquier algoritmo a fallar siempre: el resultado depende también de las víctimas elegidas, como muestra Aging con 9 marcos.

9. Preguntas frecuentes

¿Por qué Second Chance se llama "reloj"? Porque los marcos se recorren en círculo con un puntero, como las manecillas de un reloj.

¿Qué pasa si todos los bits de referencia están en 1? Los limpia y expulsa el marco donde comenzó. Si ocurre en cada reemplazo, avanza como FIFO; no implica siempre expulsar la carga más antigua.

¿Por qué Second Chance puede sufrir la anomalía de Belady? No garantiza la propiedad de pila. La traza de Belady lo demuestra: 9 fallos con 3 marcos y 10 con 4.

¿Aging es lo mismo que el aging de planificación de CPU? No. Aquí es el algoritmo de contadores de envejecimiento para reemplazo de páginas; el de CPU sube prioridades para evitar inanición.

¿Cuál es la diferencia entre Aging y LRU? LRU recuerda el instante exacto del último uso; Aging recuerda un historial pesado de las últimas N referencias.

¿Por qué Aging usa un desplazamiento a la derecha? Porque cada referencia "envejece" el historial: los bits viejos pierden peso y el uso actual entra por el MSB.

¿Por qué Aging de 8 bits pierde ventaja con más marcos? La cadena no cambia. Más marcos permiten retener páginas fuera de la ventana de historia; sus contadores empatan en cero y se pierde el orden de recencia.

¿Cómo se arregla? Aumentando los bits (14–16), o usando un algoritmo con más historia (LRU exacto, WSClock, reloj con listas activas/inactivas).

¿Por qué con 4 marcos Aging empata con LRU? Porque la ventana de 8 referencias alcanza para distinguir las páginas del conjunto de trabajo de 4 marcos.

¿Por qué OPT no se puede usar en un sistema real? Porque necesita conocer la cadena futura de referencias; el sistema operativo no la tiene.

¿Cómo saben que la simulación está bien? 24 self-tests con valores de libro, 1440 comparaciones contra Python y verificación de invariantes en cada corrida.

¿Por qué 4 marcos por defecto? Es un tamaño pequeño que hace visible la diferencia entre algoritmos y permite comparar con los ejemplos de libro; se puede cambiar con `-m`.

¿Qué pasa con 1 marco? Todos los algoritmos fallan en cada referencia que cambie de página; es el caso degenerado.

¿Cuál elegirían para un sistema real? Compararíamos fallos, costo de muestreo y metadatos con cargas reales. Aquí no se midieron tiempo de CPU ni E/S.

¿Por qué la traza cíclica da 100 fallos con 4 marcos? Las políticas en línea probadas expulsan las páginas antes de reutilizarlas. OPT da 70 fallos; con 9 marcos Aging rompe el ciclo y da 65.

¿Por qué con 10 marcos la traza cíclica da 10 fallos? Porque las 10 páginas caben y, después de la primera vuelta, todas las referencias son aciertos.

10. Conclusiones

- **Second Chance** es la opción de bajo costo: un bit por marco, un puntero y un recorrido solo en los fallos. Mejora a FIFO en las trazas con localidad y nunca es peor que FIFO en las trazas probadas. Su debilidad es que, si todos los bits están en 1 en cada reemplazo, avanza como FIFO, y no es un algoritmo de pila: exhibe la anomalía de Belady (9→10 fallos al pasar de 3 a 4 marcos).
- **Aging** consigue la mejor tasa de fallos de los dos algoritmos seleccionados con 4 marcos: empata con LRU en las trazas con localidad y reduce los fallos de Second Chance en 3–4 %. Su debilidad es el costo y la dependencia del ancho del contador: con 8 bits y memorias de 9–12 marcos se queda atrás de Second Chance y de LRU; con 14–16 bits se acerca de nuevo a LRU.
- **No hay un ganador absoluto.** El resultado depende de la localidad de la carga, del tamaño de la memoria y del presupuesto de metadata. Un algoritmo de reemplazo no se elige solo por su nombre, sino por el contexto en el que va a correr.

- **OPT es una cota, no una opción.** La diferencia entre OPT y los algoritmos reales (586 contra 945 fallos en `localidad` con 4 marcos) muestra que todavía hay margen de mejora, pero también que las aproximaciones de bajo costo se quedan lejos del óptimo teórico.
- **La validación fue parte del trabajo.** Los 24 self-tests y las 1440 comparaciones contra una implementación independiente permiten afirmar que las diferencias observadas son del algoritmo y no de un error de implementación.

Referencias

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. Capítulo 10, *Virtual Memory*, §10.4 *Page Replacement*.

[2] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson. Capítulo 3, *Memory Management*, §3.4 *Page Replacement Algorithms*.

[3] Gottlieb, A. (2009). *OS Lecture #20*, sección "The Aging PRA", New York University. `https://cs.nyu.edu/~gottlieb/courses/2000s/2008-09-spring/os202/lectures/lecture-20.html`

[4] Stallings, W. (2017). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson. Capítulo 8, *Virtual Memory*.

[5] Belady, L. A. (1966). A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal, 5*(2), 78–101.

[6] Enunciado de la Tarea 5, *Simulación de algoritmos de administración de memoria*, Sistemas Operativos.

[7] Documentación de Python 3 y Matplotlib, usadas para generar las trazas, las gráficas y la verificación cruzada: `https://docs.python.org/3/` y `https://matplotlib.org/`.